Predicting an income above 50K from census data: an end-to-end supervised learning proof of concept

Eduardo Pepe Louis Pimont-Nishimura Jules Faucher

MSA-DATB09-04 Supervised Learning & Model Engineering, Albert School
Instructor: Ilana Sebag

October 2026

Dataset: Adult / Census Income, OpenML (`name="adult"`, `version=2`).
Code: `https://github.com/epepe-10/supervised-learning-project`

1 Introduction

We predict whether a person earns more than $50,000 a year (>50K) from their census record: age, education, job, working hours, household situation and capital income. The data is the Adult dataset, extracted from the 1994 US census: 48,842 people, 14 features and a binary target in which only 23.9% of people are positive. The task is therefore a binary classification on tabular data with an imbalanced target.

The goal is a complete proof of concept, from the raw table to a tuned, explained and calibrated model evaluated on an untouched test set. The whole project follows one validation protocol. The data is split once into a development set (80%) and a test set (20%). Every decision (preprocessing, features, model, hyperparameters, imbalance remedy and decision threshold) is made with 5-fold stratified cross-validation on the development set only. The test set is used once, at the very end (Section 11).

The report follows the project requirements in order (Sections 2 to 11). The fairness and bias audit (requirement 10) was removed from the scope of the project by the instructor, so it is not covered. The code, and the order in which to run the scripts, are described in the repository's `README.md`. Supporting figures, tables and equations are in the appendix.

2 Exploratory analysis and problem framing

2.1 Data integrity

The raw table has 48,842 rows and 14 features: 6 integers and 8 categories. We checked the meaning of each column, not only its storage type, which led to three cleaning decisions.

- **Missing values** appear only in three categorical columns: `workclass` (5.7%), `occupation` (5.8%) and `native-country` (1.8%). They are imputed inside the Pipeline (Section 3), never before the split.
- **52 exact duplicate rows** (0.1%) are dropped before the split, so the same record cannot end up on both sides of a split.
- **Two columns are not given to the model.** `education` and `education-num` map one-to-one (16 values each); we keep `education-num`, which also carries the order of the levels. `fnlwgt` is a sampling weight of the census survey: it describes how the survey was designed, not the person whose income we predict.

2.2 Validation protocol

Each row is one person and there is no time dimension, so a random split is appropriate. We split the cleaned data (48,790 rows) once, stratified on the target, into a development set of 39,032 rows and a test set of 9,758 rows. Both contain 23.9% of >50K. Inside the development set, every comparison uses the same 5-fold `StratifiedKFold` (shuffled, seed 42), so all models are scored on exactly the same folds. Everything that relates the features to the target, including the analysis below, was done on the development set only.

2.3 Target and choice of metrics

A model that always predicts ≤50K is already 76.1% accurate, so accuracy alone says little. We report the same metrics for every model: ROC-AUC and Average Precision (AP, the area under the precision-recall curve), which judge how well people are ranked, without a threshold; log-loss, which judges the probabilities; and precision, recall, F1 and the confusion matrix, which judge the final decisions at a given threshold. AP is our selection metric: it focuses on how well the minority class is retrieved, and its reference value is the prevalence (0.239), against 0.5 for ROC-AUC.

2.4 Main findings

People earning >50K are older, more educated and work more hours (Table 1; full distributions in Figure 4, appendix). `capital-gain` and `capital-loss` are zero for 91.8% and 95.2% of people, with a long tail of large values. The share of >50K rises with age, peaks around 50 and then falls (Figure 1): a curve that a linear model cannot draw with `age` alone. Among the categorical features (Figure 5, appendix), the household variables separate the classes most: the share of >50K ranges from 5% to 45% across `marital-status` values and from 2% to 47% across `relationship` values. It is 30% for men and 11% for women.

Class	age	educ.-num	cap.-gain	cap.-loss	hours
≤50K	36.9	9.6	144	54	38.8
>50K	44.3	11.6	4,096	200	45.5

Table 1: Mean of each numerical feature by class (development set).



Figure 1: Share of >50K by age: an inverted U, which motivates `age_squared`.

2.5 Leakage audit and open questions

For each feature, we asked whether its value would be known when the prediction is made (Table 8, appendix). Only `capital-gain` and `capital-loss` are a medium risk: declared for the same tax year as the income, they might only become available with the income itself. We keep them, since the census collects them with the other answers, but this must be checked before any deployment. Two assumptions remain open: missing `workclass` and `occupation` values may describe people who do not work (tested in Section 12), and the data dates from 1994, so the model should not be used on recent data without retraining.

3 Feature engineering and preprocessing

3.1 Three engineered features

A logistic regression is linear in the log-odds: it cannot draw a curve, a threshold or an interaction on its own. We gave it these shapes explicitly, with one feature of each kind, each motivated by the

analysis above.

- `age_squared` = age^2 (square term). With `age` and `age_squared`, the model can draw the inverted U of Figure 1 instead of a straight line.
- `education_x_hours` = education-num × hours-per-week (interaction). The value of an extra hour of work is likely to depend on the level of education.
- `has_capital` = $\mathbf{1}$[capital-gain > 0 or capital-loss > 0] (indicator). With more than 90% of the amounts at zero, owning capital at all is a cleaner signal than the raw, skewed amounts.

The three features only combine columns of the same row. They learn nothing from the data (no mean, no median, no target), so they cannot leak information from validation rows into training rows. They are computed inside the Pipeline, so they are recomputed for any new row.

What they bought. On the untuned logistic regression (5-fold out-of-fold predictions, threshold 0.5), the three features raise ROC-AUC from 0.905 to 0.913, AP from 0.763 to 0.784 and F1 from 0.657 to 0.673: about seven times the standard deviation of AP across folds (0.003). We kept them for all three models so that every model sees the same inputs; SHAP shows that LightGBM still uses two of them (Section 9).

3.2 Missing values and encoding

All the steps below are learned on the training rows only, inside each cross-validation fold (Section 6).

- **Categorical columns** (7): missing values are replaced by the most frequent category, then one-hot encoded, since categories are labels, not measurements. `handle_unknown="ignore"` turns a category never seen in training into a row of zeros instead of an error at prediction time.
- **Numerical columns** (5 original and 3 engineered): median imputation, then standardization $x \mapsto (x - \mu)/\sigma$, which matters for the regularized logistic regression. Adult has no missing numerical value, but the imputer stays so that a new row with a missing number can still be scored.

After encoding, the model sees 91 columns. Section 12 tests a separate "missing" category instead of the most frequent one.

4 Class imbalance

With 23.9% of positives, a model can look accurate while missing most of the >50K people. We compare three ways of handling the imbalance, from the least to the most invasive, on the three untuned models (Table 2).

- **No remedy**: the model as it is, with the default threshold 0.5.
- **Class weights**: `class_weight="balanced"` gives both classes the same total weight, so each >50K person weighs $N_{\text{neg}}/N_{\text{pos}} \approx 3.2$ times a ≤50K person (computed on the training folds only). The model is retrained, so its probabilities change.
- **Threshold tuning**: the model without weights, but the cut-off 0.5 is replaced by the threshold that maximizes F1 on the out-of-fold probabilities (grid 0.05 to 0.95, step 0.01). Only the decisions change. We maximize F1 because the dataset comes with no business cost for each type of error, and F1 assumes that both errors cost about the same.

We did not use SMOTE. It creates synthetic people by interpolating between neighbours, which has no meaning for our seven categorical features: there is nothing halfway between two occupations or two countries.

Table 2: Imbalance remedies on the untuned models (5-fold out-of-fold predictions on the development set). The matching confusion matrices are in Figure 6 (appendix).

Model	Remedy	AP	Log-loss	Precision	Recall	F1	Threshold
Log. regression	None	0.784	0.309	0.747	0.613	0.673	0.50
	Class weights	0.783	0.379	0.576	0.852	0.687	0.50
	Threshold	0.784	0.309	0.659	0.745	**0.699**	0.36
Random forest	None	0.750	0.446	0.711	0.624	0.665	0.50
	Class weights	0.724	0.510	0.629	0.743	**0.681**	0.50
	Threshold	0.750	0.446	0.622	0.750	0.680	0.33
LightGBM	None	0.831	0.275	0.779	0.653	0.711	0.50
	Class weights	0.830	0.332	0.605	0.867	0.713	0.50
	Threshold	0.831	0.275	0.708	0.756	**0.731**	0.38

What we learn. At the default threshold, the models find only 61% to 65% of the >50K people. Both remedies trade precision for recall, but in different ways.

- **Class weights** raise recall strongly (0.613 to 0.852 for the logistic regression) at a high cost in precision (0.747 to 0.576), for a small F1 gain. They never improve AP, and they worsen the log-loss of all three models (0.275 to 0.332 for LightGBM): the probabilities are pushed up and no longer match the observed frequencies.
- **Threshold tuning** gives the best F1 for the logistic regression (0.699) and LightGBM (0.731), and ties for the random forest, without retraining or touching the probabilities. The thresholds (0.33 to 0.38) are well below 0.5, as expected with a minority class. This F1 is slightly optimistic (threshold chosen and scored on the same predictions), but with one parameter and 39,032 rows the bias is small.

Chosen remedy. We keep both levers, each in its place. The class weight becomes a hyperparameter that Optuna may use or not (Section 7): `class_weight` $\in$ {None, balanced} for the logistic regression and the random forest, and `scale_pos_weight` $\in [1, 3.18]$ for LightGBM. Then the decision threshold is chosen on the out-of-fold probabilities of each tuned model and frozen before the test set is used. Section 10 comes back to the price of class weighting on the probabilities.

5 Modelling progression

We compared three models of increasing flexibility, inside the same Pipeline, on the same folds and with the same metrics. Their equations (sigmoid and penalized log-loss, Gini and entropy splits, gradient boosting update) are in Appendix A.

1. **Logistic regression**: linear in the log-odds, interpretable and probabilistic, which makes it a strong baseline. The `saga` solver supports the three penalties: `l1_ratio` sets the mix (0 = Ridge, 1 = Lasso, in between = Elastic Net) and $C = 1/\lambda$ its strength.
2. **Random forest**: many deep trees, each trained on a bootstrap sample with a random subset of features at each split, then averaged. Averaging decorrelated trees mainly reduces the variance.
3. **LightGBM**: gradient boosting. Small trees are added one at a time, each fitted to the errors of the current ensemble, which mainly reduces the bias. We chose LightGBM over XGBoost or CatBoost, which implement the same algorithm, because it is the fastest, which matters when each tuning trial means five fits; CatBoost's strength, high-cardinality categories, does not apply here (`native-country`, the largest, has about forty).

5.1 Untuned models

Table 3: Untuned models, default hyperparameters (5-fold out-of-fold predictions on the development set). "AP std" is the standard deviation of AP across the 5 folds. ROC and PR curves: Figure 7 (appendix).

Model	ROC-AUC	AP	AP std	Log-loss	F1 (0.5)	F1 (tuned thr.)
Log. reg., no new features	0.905	0.763	0.002	0.320	0.657	–
Logistic regression	0.913	0.784	0.003	0.309	0.673	0.699
Random forest	0.895	0.750	0.006	0.446	0.665	0.680
LightGBM	**0.929**	**0.831**	0.001	**0.275**	**0.711**	**0.731**

What each step bought. Feature engineering added 0.021 AP to the logistic regression (Section 3). The untuned random forest is *worse* than the logistic regression (AP 0.750 against 0.784, log-loss 0.446): with the default settings, its trees grow until each leaf is pure, so they memorize the training folds and the averaged probabilities are poorly calibrated. LightGBM is the best model on every metric: AP 0.831 (+0.047 over the logistic regression), ROC-AUC 0.929, log-loss 0.275 and F1 0.731. The gaps between models (at least 0.03 AP) are about ten times the variation of AP across folds (at most 0.006), so the ranking is not an artefact of the split.

5.2 Tuned models

Table 4: Tuned models (5-fold out-of-fold predictions on the development set, threshold chosen on these predictions to maximize F1).

Model	ROC-AUC	AP	Log-loss	Precision	Recall	F1	Threshold
Logistic regression	0.913	0.784	0.309	0.665	0.736	0.699	0.37
Random forest	0.921	0.814	0.343	0.670	**0.773**	0.718	0.62
LightGBM	**0.930**	**0.832**	**0.287**	**0.718**	0.748	**0.733**	0.54

What tuning bought. Tuning mattered most where the defaults were worst. The random forest gains 0.064 AP (0.750 to 0.814) and 0.038 F1 once its depth and leaf size are limited: it stops memorizing the training folds and moves from last to second place. This is the bias–variance trade-off: a deep tree with tiny leaves has low bias but high variance, and the tuned depth of 14 with at least 3 rows per leaf is the compromise chosen by cross-validation. The logistic regression does not move (AP 0.784, F1 0.699): with three hyperparameters and a linear boundary, its defaults were already close to its best (Section 7.2). LightGBM gains almost nothing (AP 0.831 to 0.832), because its defaults were already well suited to this table. The ranking is the same on every metric except recall, where the balanced random forest is ahead. The final model is the one with the best out-of-fold AP after tuning, LightGBM, chosen before the test set is used.

6 A single Pipeline

All the preprocessing and the model are wrapped in one scikit-learn `Pipeline`, which takes a raw census row and returns $P(>50\text{K})$: (1) a stateless `FunctionTransformer` adds the three engineered features; (2) a `ColumnTransformer` applies median imputation and `StandardScaler` to the numerical columns, most-frequent imputation and `OneHotEncoder(handle_unknown="ignore")` to the categorical ones, and drops `education` and `fnlwgt` (`remainder="drop"`); (3) the classifier. One Pipeline brings four guarantees.

- **No leakage during cross-validation.** A fresh, unfitted copy (`clone`) is fitted in each fold, so the medians, means, standard deviations and categories are learned on the four training folds only. Imputing or scaling the whole development set first would have let each validation fold influence its own preprocessing.

- **The same transformations everywhere.** The same object is cross-validated, refitted on the whole development set, evaluated on the test set and saved: no second implementation of the preprocessing can drift from the first one.
- **Tuning covers the whole chain.** Each Optuna trial builds and cross-validates a complete Pipeline (Section 7).
- **One file to deploy.** The fitted Pipeline is saved with `joblib`, together with its decision threshold, since both are needed to turn a raw row into a decision. It also returns the name of each encoded column (`get_feature_names_out`), which the SHAP analysis uses.

7 Hyperparameter tuning

Protocol. We tuned each model with Optuna and its TPE sampler (seed 42, so a re-run proposes the same trials). In each trial, Optuna proposes a set of hyperparameters, we build the complete Pipeline with them, score it with the same 5-fold stratified cross-validation on the development set, and return the mean AP over the 5 folds; TPE uses all past trials to choose the next configuration. The budget is 25 trials for the logistic regression, 20 for the random forest (the slowest to fit) and 50 for LightGBM, which has the largest search space. A grid search would not fit this budget: three values for each of LightGBM's eight hyperparameters already make $3^8 = 6{,}561$ configurations, i.e. about 33,000 fits. A random search would fit it, but ignores past scores, whereas TPE spends more trials in the promising regions. The search spaces (Table 9, appendix) include the class weighting, from none up to the class ratio (Section 4). We optimize AP because it judges the ranking of the minority class without depending on a threshold, which is chosen afterwards.

Why the best CV score is optimistic. The best of 20 to 50 trials is selected on the same folds that score it, so part of its advantage is luck on these folds. We therefore do not report it as our final result: the honest estimate comes from the test set, which played no role in tuning (Section 11). Nested cross-validation would remove this bias without a test set, but would multiply the cost of tuning by five; with 48,790 rows, a held-out test set was the cheaper option.

Final training. For each model, `train.py` (1) computes the out-of-fold probabilities of the tuned Pipeline on the same 5 folds, (2) chooses the threshold that maximizes F1 on them and freezes it, (3) refits the Pipeline on the whole development set, and (4) saves it with its threshold.

7.1 Hyperparameters selected by Optuna

Table 5: Best hyperparameters found by Optuna and best mean CV AP. What each LightGBM hyperparameter controls is explained in Table 10 (appendix).

Model	Best hyperparameters	CV AP
Logistic regression	$C = 1.89$, `l1_ratio` = 0.77 (Elastic Net, mostly Lasso), no class weight	0.784
Random forest	200 trees, `max_depth` = 14, `min_samples_leaf` = 3, `max_features` = 0.60, `class_weight="balanced"`	0.815
LightGBM	350 trees, learning rate 0.048, 26 leaves, `min_child_samples` = 11, `subsample` = 0.81, `colsample_bytree` = 0.57, $\lambda = 0.015$, `scale_pos_weight` = 1.80	0.833

- **Edges of the ranges.** Only `max_features` = 0.60 for the random forest landed on a bound; we did not extend it, since the forest stays well behind LightGBM.
- **Class weighting.** No weight for the logistic regression, `"balanced"` for the forest, 1.80 for LightGBM: since the threshold is tuned afterwards, the weight is one more hyperparameter, kept when it helps AP.

- **A flat optimum for LightGBM.** Its best trial is trial 2: the 47 following trials did not beat it (Figure 8, appendix). Below a learning rate of 0.02, even 1,000 trees do not finish learning (mean AP 0.811 over 2 trials); above 0.1, each tree corrects too much at once (0.826 over 21 trials); between the two, the 27 trials average 0.829 and the ten best lie within 0.002 AP, about the variation across folds. The selected $\eta = 0.048$ with 350 trees is one good point of a plateau, and more trials would not change the model much.

7.2 What the penalty does to the logistic regression

The regularization path (train and validation AP for C from 10^{-4} to 100, `l1_ratio` = 0.77; Figure 9, appendix) shows three things. **Sparsity:** the L1 part of the tuned penalty sets 9 of the 91 coefficients exactly to zero (seven rare `native-country` categories, `workclass` = Never-worked, `relationship` = Not-in-family), where Ridge would only shrink them. **Strong penalty = underfitting:** at $C = 10^{-4}$ every coefficient is zero and the model predicts the base rate (AP 0.24). **Weak penalty does not overfit here:** above $C \approx 0.1$ the validation AP is flat (0.782 to 0.784) and the train–validation gap stays below 0.004 even at $C = 100$, since 39,000 rows cannot be memorized by 91 coefficients. The tuned $C = 1.89$ lies in this flat region, which is why tuning did not improve the model; $C = 0.1$ would give a shorter model (39 coefficients) for only 0.002 AP less.

8 Experiment tracking

Every tuning trial and every final training is logged with MLflow, in a local database (`mlflow.db`, with the files in `mlruns/`), under a single experiment, `adult-income`:

- **one parent run per tuning study** (e.g. "lightgbm - tuning"): the best hyperparameters, the best mean CV AP, and the tuning files as artifacts (best parameters, table of all trials, optimisation history);
- **one child run per Optuna trial**: its hyperparameters, the AP of each of the 5 folds, their mean and standard deviation. Logging the five folds, not only the mean, shows how much a score depends on the split;
- **one run per final training**: the tuned hyperparameters, the out-of-fold metrics, the chosen threshold and the saved model as an artifact.

In total, the 95 tuning trials (25 + 20 + 50), the 3 tuning studies and the 3 final trainings are logged (Figure 12, appendix). Since the sampler and the folds are seeded, re-running the tuning with tracking switched on proposes the same trials: the best hyperparameters, the frozen thresholds and the out-of-fold scores of the three models were identical after the re-run.

9 Explainability with SHAP

Method. SHAP splits one prediction into a reference value plus one contribution per feature: $f(x) = \phi_0 + \sum_j \phi_j(x)$. For LightGBM, `TreeExplainer` computes these values exactly, in log-odds of >50K. We explain the final model on 1,000 people drawn from the development set (the test set stays untouched). SHAP values are additive, so we sum the values of the one-hot columns of each categorical feature to get one contribution per original feature. As a check, for every explained person, the base value plus the contributions gives back the raw score, and its sigmoid the predicted probability.

Global importance. The global importance of a feature is its mean absolute SHAP value (Figure 2). Marital status comes first by far (0.93 in log-odds), followed by age (0.58), capital gains (0.53), relationship (0.41) and our interaction `education_x_hours` (0.39); occupation (0.31) and `education-num` (0.30) come next. Sex is 9th (0.18) and race 14th (0.06) out of 15. The beeswarm adds the direction: a large capital gain pushes the score up by +4 to +7, the largest single effects of the model, and young ages pull it down (to about −2.5).

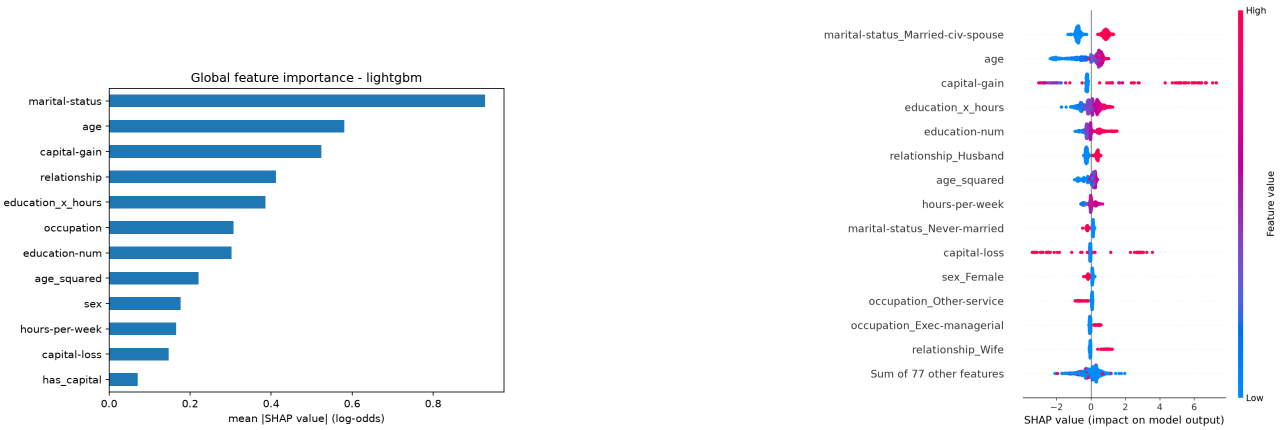


Figure 2: Global SHAP importance per original feature (left) and beeswarm plot per encoded column (right) of the final model, on 1,000 people of the development set.

Local explanations. We explain three people, all starting from the base value $\phi_0 = -1.93$ (a probability of 0.13; waterfall plots in Figure 13, appendix). The most confident >50K prediction ($P = 0.999$, truly >50K) owes +5.67 of its +9.26 to a capital gain of \$15,024, the rest to marriage, 16 years of education and an executive job. The most confident ≤50K ($P < 0.001$, truly ≤50K) is mainly explained by age 18 (−2.17 for `age`, −0.79 for `age_squared`). The person closest to the threshold ($P = 0.539$, just below 0.54, truly ≤50K) is a married man of 35: marriage (+0.88) and a high `education_x_hours` (+0.63) push up, but the absence of capital gains (−0.21) keeps him under the threshold.

Do the important features make business sense? Mostly yes. The household variables come first (marital status and relationship together weigh 1.34): being married is a proxy for age, career stage and a stable household. Capital gains are a direct sign of wealth, and age, education and occupation describe a career. Our engineered features are used: `education_x_hours` is 5th, ahead of `education-num`, and `age_squared` 8th, while `has_capital` is only 12th because the trees can split on `capital-gain` directly. Sex weighs less, but not zero: being a woman lowers the score by about 0.3 in log-odds, on top of what passes through `relationship`. The model reproduces the income gap between men and women in the 1994 census; with the fairness audit out of scope, we flag it as a limitation. Two cautions apply: SHAP explains what the model does, not what causes a high income, and correlated features (`age` and `age_squared`, `marital-status` and `relationship`) share the credit and must be read together.

10 Calibration diagnostics

Ranking and calibration are two different questions. AP and ROC-AUC ask whether >50K people get higher scores than the others; calibration asks whether a predicted 0.7 means that about 70% of such people really earn >50K.

Method. We judge the final model on its out-of-fold probabilities on the development set: each person is predicted by a model that did not train on them, and the test set stays untouched. The reliability diagram groups the predictions into 10 equal-width bins and compares, in each bin, the mean predicted probability with the observed share of >50K. Two scores complete it (lower is better): the **Brier score**, the mean squared error between the predicted probability and the 0/1 outcome (always predicting the prevalence gives $0.239 \times 0.761 = 0.182$), and the **expected calibration error**, $\text{ECE} = \sum_m \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)|$, the gap between observed share and mean prediction in each bin B_m, weighted by its share of people.

Results. On the 39,032 out-of-fold predictions of LightGBM, the Brier score is 0.092, half of the 0.182 of the constant prediction, and the ECE is 0.049 (Figure 3; per-bin values in Table 12, appendix). Below 0.1, where half of the people fall, the model is nearly calibrated; around 0.55, it predicts 55% where only 39% earn >50K. The mean prediction is 0.289 against an observed share of 0.239.

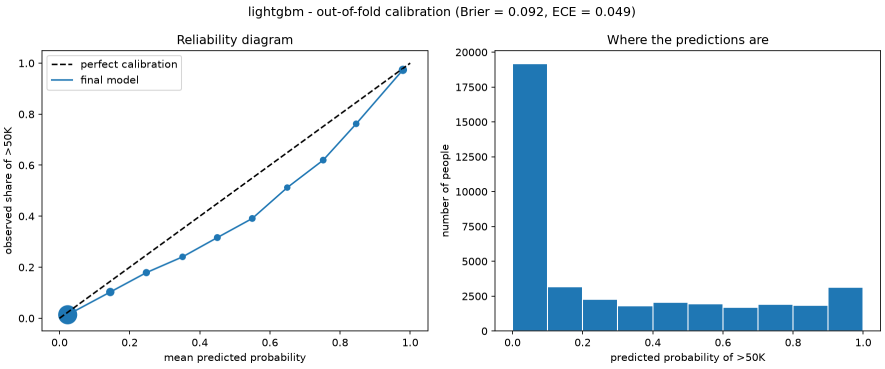


Figure 3: Reliability diagram (left) and histogram of the predicted probabilities (right) of the final model, out-of-fold on the development set.

Over- or under-confident? The curve lies below the diagonal, with a positive gap in all ten bins (from +0.005 to +0.159): the model is over-confident in its >50K predictions, most of all in the middle of the range.

Why: the class weight. Optuna kept `scale_pos_weight` = 1.80 for LightGBM, which makes each >50K person count more during training and shifts all the probabilities up. To check this, we compared two untuned LightGBM models that differ only by this weight (Table 6).

Table 6: Untuned LightGBM with and without class weight (5-fold out-of-fold, development set). The observed share of >50K is 0.239.

`scale_pos_weight`	AP	F1 (tuned thr.)	Brier	ECE	Mean prediction
1.0	0.8307	0.7314	0.0876	0.0046	0.239
1.8	0.8307	0.7307	0.0931	0.0511	0.291

The weight changes neither the ranking (same AP) nor the decisions (same F1), but it multiplies the ECE by eleven: AP only depends on the order of the scores, so Optuna had no reason to prefer one weight over another.

Would we trust the probabilities as-is? For the decisions, yes: the threshold was chosen on these same, shifted probabilities, so the >50K / ≤50K decisions are consistent. For the probabilities themselves, no: any downstream use that reads them as real chances would over-estimate. Rather than recalibrating after training (Platt scaling or isotonic regression), we would fix `scale_pos_weight` to 1: it costs nothing in ranking or decisions and brings the ECE from 0.051 to 0.005 (Table 6). We did not apply it because the test set had already been used (Section 11): changing the model afterwards would have required a second look at it.

11 Final evaluation on the untouched test set

The three tuned models were evaluated once on the 9,758 people of the test set, each with the threshold frozen during training (Section 7). Nothing was changed after this evaluation.

Test against cross-validation. The test scores (Table 7) are slightly below the out-of-fold scores of Table 4: about −0.01 AP and −0.01 F1 for every model (LightGBM: AP 0.832 to 0.821, F1 0.733 to 0.722). This small drop is the one expected from selecting hyperparameters and thresholds on the same folds (Section 7), and the ranking is unchanged. LightGBM reaches 86.3% accuracy (76.1% for the trivial model) and finds 74% of the >50K people with a precision of 70% (confusion matrices and curves in Figures 11 and 10, appendix).

Table 7: Test-set results of the three tuned models (thresholds frozen on the development set).

Model	ROC-AUC	AP	Log-loss	Accuracy	Precision	Recall	F1	Thr.
Logistic regression	0.910	0.773	0.317	0.845	0.658	0.732	0.693	0.37
Random forest	0.918	0.805	0.353	0.847	0.651	**0.778**	0.709	0.62
LightGBM	**0.926**	**0.821**	**0.297**	**0.863**	**0.702**	0.744	**0.722**	0.54

12 Alternatives tested, limitations and conclusion

12.1 An alternative we tested: a "missing" category

Are the missing job values random (Section 2)? On the development set, the 2,233 people with a missing `workclass` all have a missing `occupation` too, and only 9.4% of the people with a missing occupation earn >50K, against 23.9% overall: they are mostly people without a job, whom the most-frequent imputation gives the most common job. A separate "missing" category keeps this information, but on the untuned models it gains only 0.003 AP for the logistic regression and 0.001 for LightGBM, with the same F1 (Table 11, appendix): of the order of the fold-to-fold variation, so we kept the most-frequent imputation rather than re-run the tuning; a separate category remains more principled in production.

12.2 Overfitting: what we checked and what we did not

Overfitting the validation folds: checked. The 95 trials and the thresholds were chosen on the same 5 folds, so the best CV scores are optimistic by construction. The test set measures by how much: every model loses about 0.01 AP (LightGBM 0.832 to 0.821) and the ranking does not change. **Memorizing the training rows: checked for the logistic regression only**, whose train–validation gap stays below 0.004 (Section 7.2). For the tree models we never computed a training score; the evidence is indirect (limiting the forest's depth and leaf size raised its CV AP by 0.064, and the test scores hold). A table of train, validation and test AP, and a LightGBM learning curve, would measure the gap.

12.3 Limitations

Old data: the census dates from 1994, so the model would need retraining on recent data. **Possible leakage:** `capital-gain` and `capital-loss` are declared for the same tax year as the income. **F1 stands in for business costs:** with real costs per error, the threshold should minimize the expected cost. **Threshold chosen on fold models** trained on 80% of the development set, then applied to the refitted model. **Probabilities not calibrated** as-is (Section 10). **No fairness audit:** the model uses `sex` and `race`, and the share of >50K is 30% for men against 11% for women; the audit was out of scope, but would be needed before any real use.

12.4 Conclusion

Our final model, a tuned LightGBM in one Pipeline, reaches an AP of 0.821 and an F1 of 0.722 on the untouched test set, within 0.01 of cross-validation. What mattered most was the model (+0.05 AP over the logistic regression), then the decision threshold, then the engineered features. Its decisions are reliable; its probabilities, inflated by the class weight, need recalibrating (Section 10).

Contribution statement

Requirement or stage	Main author	Reviewed by
Repository, data loading and split (1)	Eduardo	Jules
EDA and problem framing (1)	Eduardo	Jules
Feature engineering (2)	Eduardo	Jules
Class imbalance (3)	Eduardo	Jules
Modelling progression (4)	Eduardo	Jules
Pipeline (5)	Eduardo	Jules
Optuna tuning, final training, test evaluation (6)	Eduardo	Jules
Experiment tracking with MLflow (7)	Louis	Eduardo, Jules
SHAP explainability (8)	Louis	Jules
Calibration diagnostics (9)	Louis	Jules
Code review and fixes, alternatives tested (Section 12)	Jules	Eduardo, Louis
Report	All three	All three

Use of AI coding assistants

We used Claude (Anthropic), through the Claude Code assistant, to draft parts of the code, to review it and suggest fixes, and to help structure and draft this report. Eduardo's and Jules's commits written with its help carry a "Co-Authored-By: Claude" line in the Git history; Louis's commits for steps 7 to 9 were also written with its help but do not carry this line. Every methodological choice was discussed by the team, every number in this report comes from running our own scripts, and each member can explain the code.

A The three models in equations

Logistic regression. The model computes a linear score and turns it into a probability with the sigmoid:

$$p(x) = \sigma(z) = \frac{1}{1+e^{-z}}, \qquad z = \beta_0 + \sum_j \beta_j x_j = \log\frac{p}{1-p}.$$

It is linear in the log-odds, not in the probability, so its decision boundary $z = \text{const}$ is a hyperplane in the feature space: curves, thresholds and interactions must be given to it as features (Section 3). The coefficients minimize the log-loss (the negative log-likelihood) plus the Elastic Net penalty:

$$\min_{\beta} \; -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right] + \lambda\Big[\alpha\|\beta\|_1 + \tfrac{1-\alpha}{2}\|\beta\|_2^2\Big],$$

with $\alpha =$ `l1_ratio` and $\lambda = 1/C$ (scikit-learn writes the same trade-off with C in front of the loss). A coefficient adds β_j to the log-odds, i.e. multiplies the odds by e^{β_j}, other columns held fixed. Since the one-hot encoding keeps both `sex` columns, the effect of being a woman rather than a man is the difference of the two coefficients: $-1.87 - (-1.13) = -0.74$, so the odds of >50K are multiplied by $e^{-0.74} \approx 0.48$ at equal age, education, job and hours. The effect is constant on the odds, not on the probability, because the sigmoid is not linear.

How a tree chooses its splits. A node is *pure* when all its rows belong to the same class. With p the share of >50K in the node, the two usual impurity measures are

$$\text{Gini} = 1 - \sum_k p_k^2 = 2p(1-p), \qquad \text{Entropy} = -\sum_k p_k \log_2 p_k.$$

Both are 0 for a pure node and largest at $p = 0.5$. At the root of our trees, $p = 0.24$, so Gini = 0.365 and entropy = 0.795 bits. At each node, the tree keeps the candidate split that most reduces the impurity, weighted by the number of rows in each child (with entropy, this reduction is the *information gain*). The two measures almost always pick the same splits; we kept scikit-learn's default, Gini, which is faster (no logarithm), and did not tune it. Left alone, the tree keeps splitting until every leaf is pure, i.e. until it has memorized its training rows, which is why the forest's `max_depth` and `min_samples_leaf` matter. LightGBM does not use an impurity measure: it picks the split that most reduces the log-loss, computed from the gradients of the loss, and its L2 penalty `reg_lambda` enters that gain directly.

How boosting corrects its errors. LightGBM builds an additive model on the log-odds scale. It starts from a constant F_0 (the log-odds of the base rate), then, at each round m, fits a small tree T_m to the negative gradient of the log-loss with respect to the current score, which is simply $r_i = y_i - p_i$: positive when the model underestimates a >50K person, negative when it overestimates a ≤50K one. The tree is added with a learning rate η:

$$F_m(x) = F_{m-1}(x) + \eta\, T_m(x), \qquad p(x) = \sigma\big(F_M(x)\big).$$

A small η makes each correction cautious and needs more trees, so the two are tuned together; our tuned model uses $\eta = 0.048$ and 350 trees. The random forest averages independent deep trees, which mainly reduces the variance, while boosting adds corrections one after the other, which mainly reduces the bias but can overfit with too many or too complex trees.

B Additional tables

Table 8: Leakage audit: could we know this value when the prediction is made?

Features	Known at prediction time	Risk	Why
age, sex, race, native-country	Yes	Low	Fixed personal characteristics
education-num	Yes	Low	Obtained before the income is earned
workclass, occupation, hours-per-week	Yes	Low	Describe the current job
marital-status, relationship	Yes	Low	Household situation
capital-gain, capital-loss	Same tax year	Medium	Declared with the income
fnlwgt	Yes	Not a person feature	Survey weight, dropped

Table 9: Search spaces of the three models. "log": sampled on a log scale. $r = N_{\text{neg}}/N_{\text{pos}} \approx 3.18$ on the development set.

Model (trials)	Hyperparameter	Range
Logistic regression (25)	`C`	$[10^{-3}, 10]$, log
	`l1_ratio`	$[0, 1]$
	`class_weight`	None or balanced
Random forest (20)	`n_estimators`	100 to 300, step 50
	`max_depth`	4 to 24
	`min_samples_leaf`	1 to 50, log
	`max_features`	$[0.1, 0.6]$ (share of the columns)
	`class_weight`	None or balanced
LightGBM (50)	`n_estimators`	100 to 1000, step 50
	`learning_rate`	$[0.01, 0.2]$, log
	`num_leaves`	8 to 128, log
	`min_child_samples`	5 to 100, log
	`subsample`	$[0.5, 1]$, applied at every iteration
	`colsample_bytree`	$[0.5, 1]$
	`reg_lambda`	$[10^{-3}, 10]$, log
	`scale_pos_weight`	$[1, r]$

Table 10: The hyperparameters selected for the final model (LightGBM) and what they control.

Hyperparameter	Value	What it controls
`n_estimators`, `learning_rate`	350, 0.048	Number of trees and weight η of each correction: many small steps rather than a few large ones (Appendix A).
`num_leaves`	26	Complexity of each tree (LightGBM grows trees leaf-wise, so it limits leaves rather than depth).
`min_child_samples`	11	Minimum number of rows in a leaf: no rule built on a handful of people.
`subsample`, `colsample_bytree`	0.81, 0.57	Share of rows and of columns each tree sees: randomness that decorrelates the trees and limits overfitting.
`reg_lambda`	0.015	L2 penalty on the leaf values; almost none was needed.
`scale_pos_weight`	1.80	Weight of a >50K person in the loss, about half of the class ratio (3.2); it shifts the probabilities up (Section 10).

Table 11: Most-frequent imputation against a "missing" category for the categorical columns (untuned models, 5-fold out-of-fold, development set).

Model	Imputation	AP	AP std	ROC-AUC	F1 (tuned thr.)
Logistic regression	Most frequent	0.7839	0.0032	0.9125	0.6992
	"missing"	0.7864	0.0039	0.9136	0.7013
LightGBM	Most frequent	0.8307	0.0014	0.9289	0.7314
	"missing"	0.8318	0.0020	0.9295	0.7313

Table 12: Reliability table of the final model (10 equal-width bins, out-of-fold on the development set). Gap = mean prediction − observed share.

Bin	Mean prediction	Observed share	Gap	People
0.0–0.1	0.023	0.014	+0.010	19,171
0.1–0.2	0.145	0.103	+0.042	3,180
0.2–0.3	0.248	0.179	+0.069	2,270
0.3–0.4	0.351	0.241	+0.110	1,824
0.4–0.5	0.450	0.316	+0.134	2,046
0.5–0.6	0.550	0.391	+0.159	1,961
0.6–0.7	0.649	0.512	+0.137	1,712
0.7–0.8	0.752	0.620	+0.132	1,923
0.8–0.9	0.846	0.762	+0.084	1,825
0.9–1.0	0.979	0.974	+0.005	3,120

C Additional figures

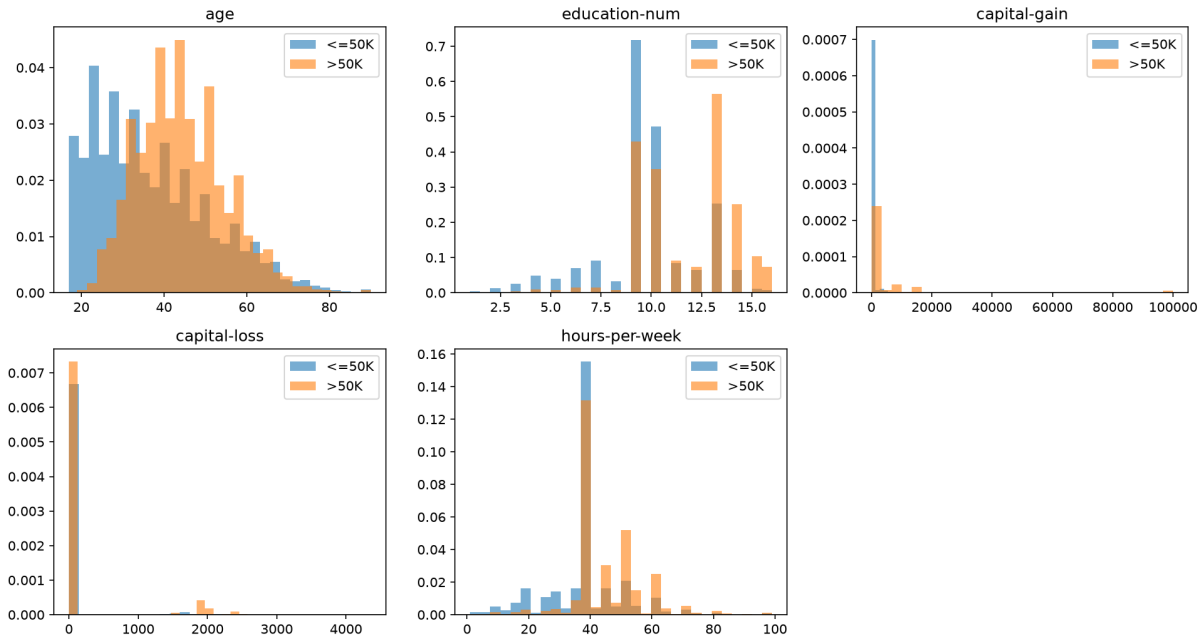


Figure 4: Distribution of each numerical feature by class (development set, densities).

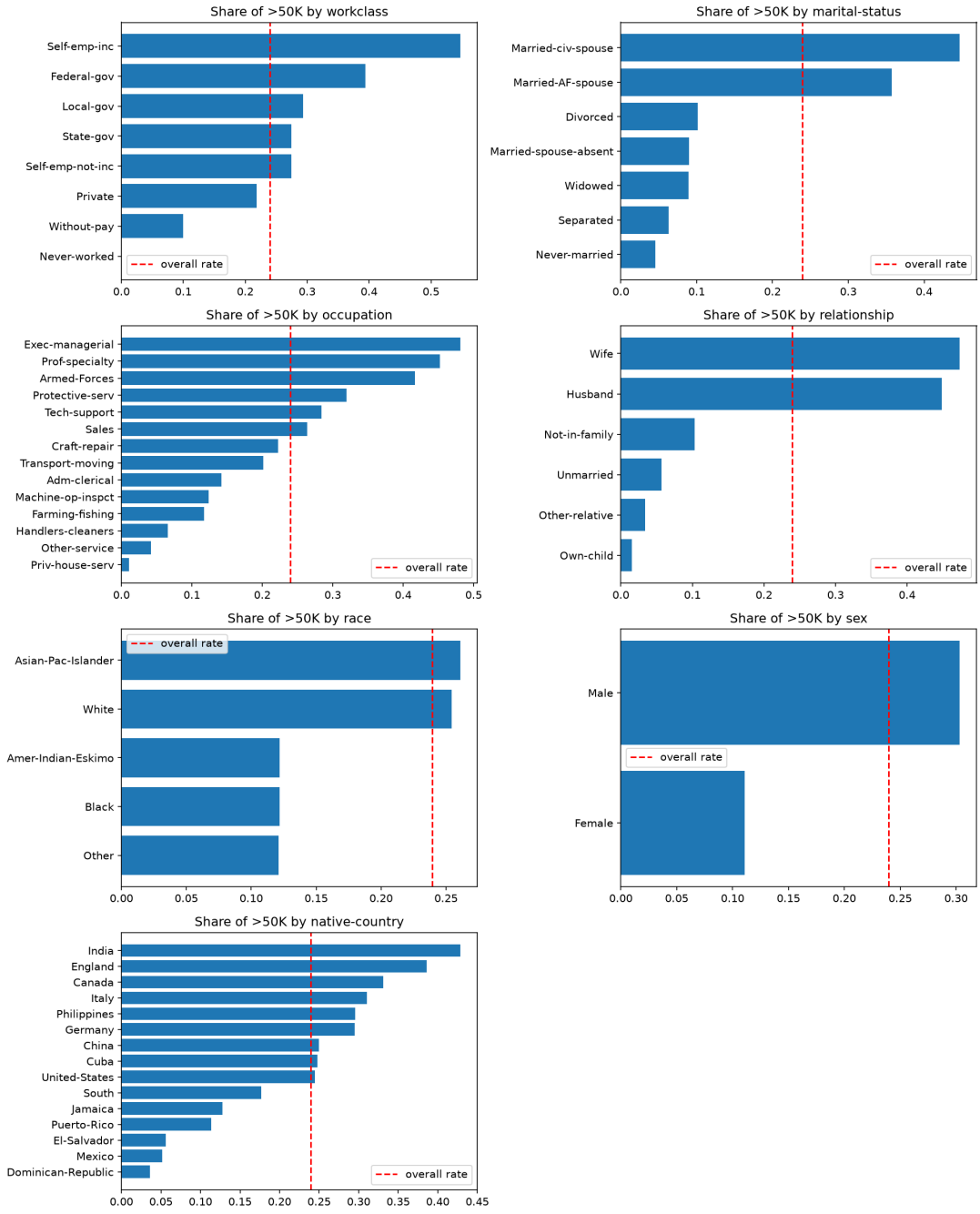


Figure 5: Share of >50K per category of each categorical feature (development set). The dashed line is the overall share. For `native-country`, only the 15 most frequent countries are shown.

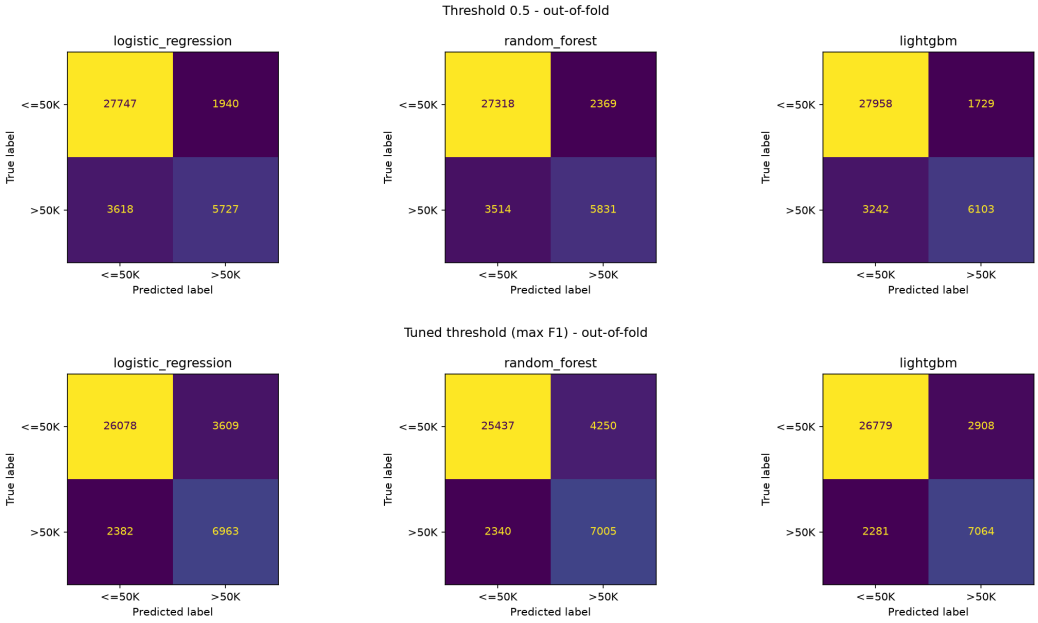


Figure 6: Out-of-fold confusion matrices of the untuned models, at the threshold 0.5 (top) and at the threshold that maximizes F1 (bottom).

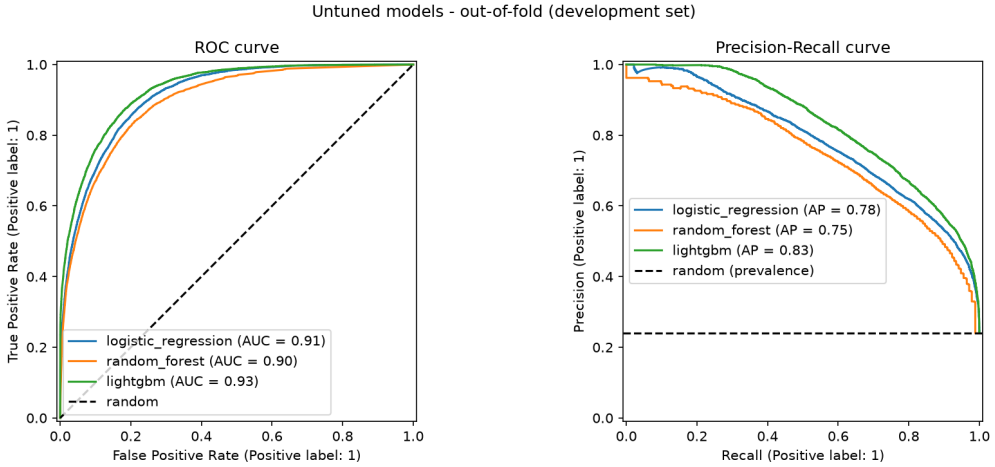


Figure 7: ROC and precision-recall curves of the untuned models (out-of-fold, development set). Dashed lines: a random model.

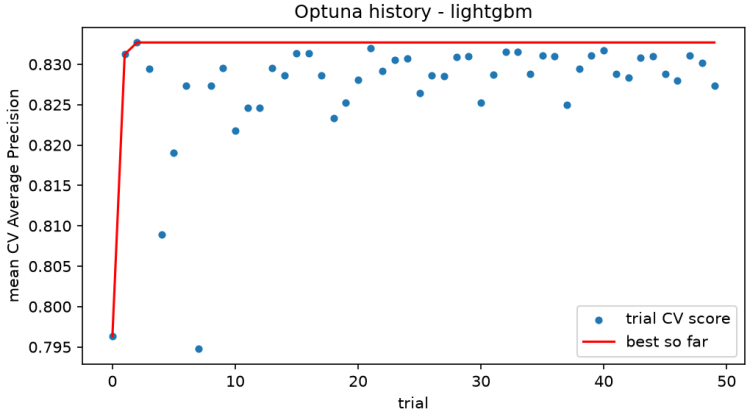


Figure 8: Mean CV AP of each Optuna trial for LightGBM.

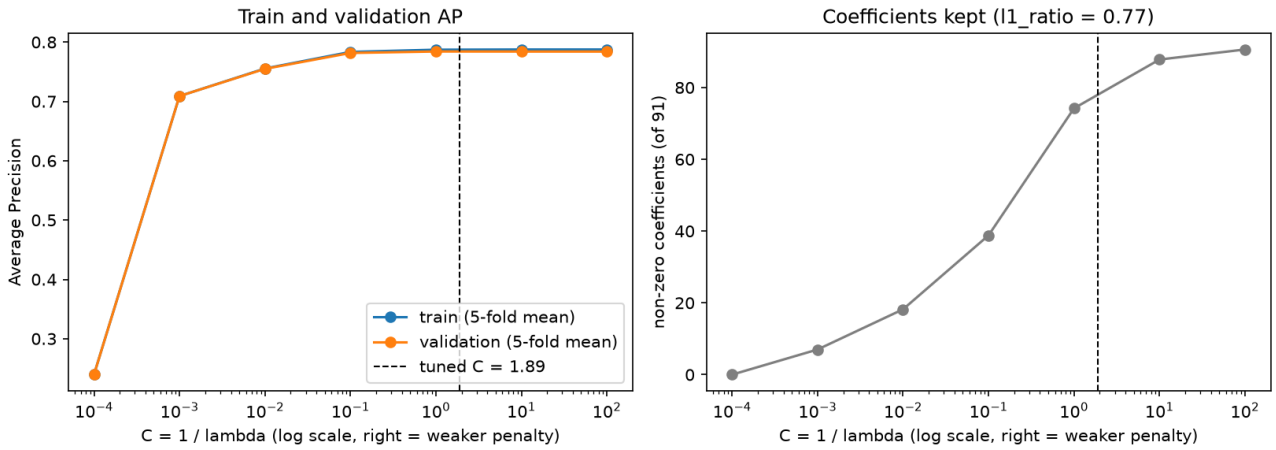


Figure 9: Regularization path of the logistic regression (`l1_ratio` = 0.77). Left: mean train and validation AP over the 5 folds. Right: mean number of non-zero coefficients. Dashed line: the tuned C.

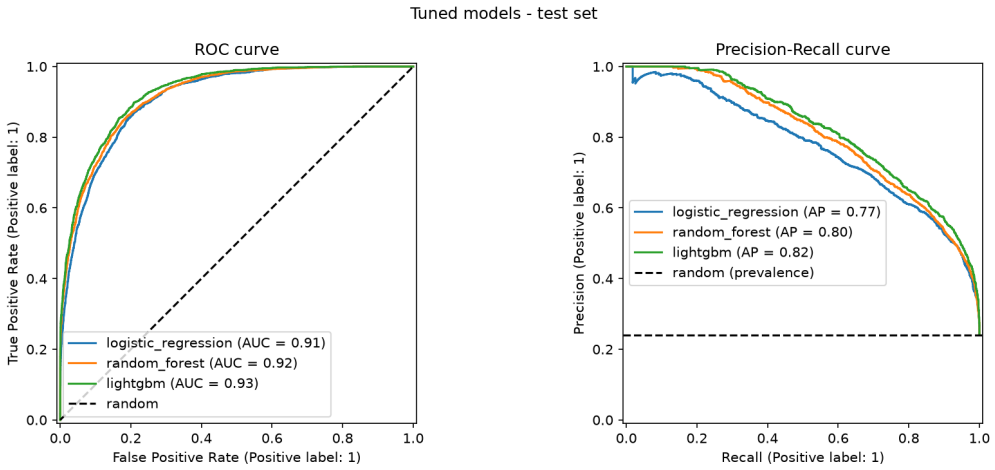


Figure 10: ROC and precision-recall curves of the three tuned models on the test set.

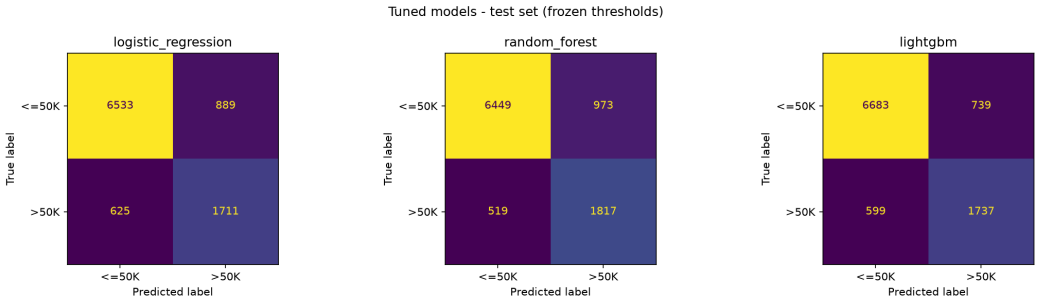


Figure 11: Confusion matrices on the test set, at the frozen threshold of each model.

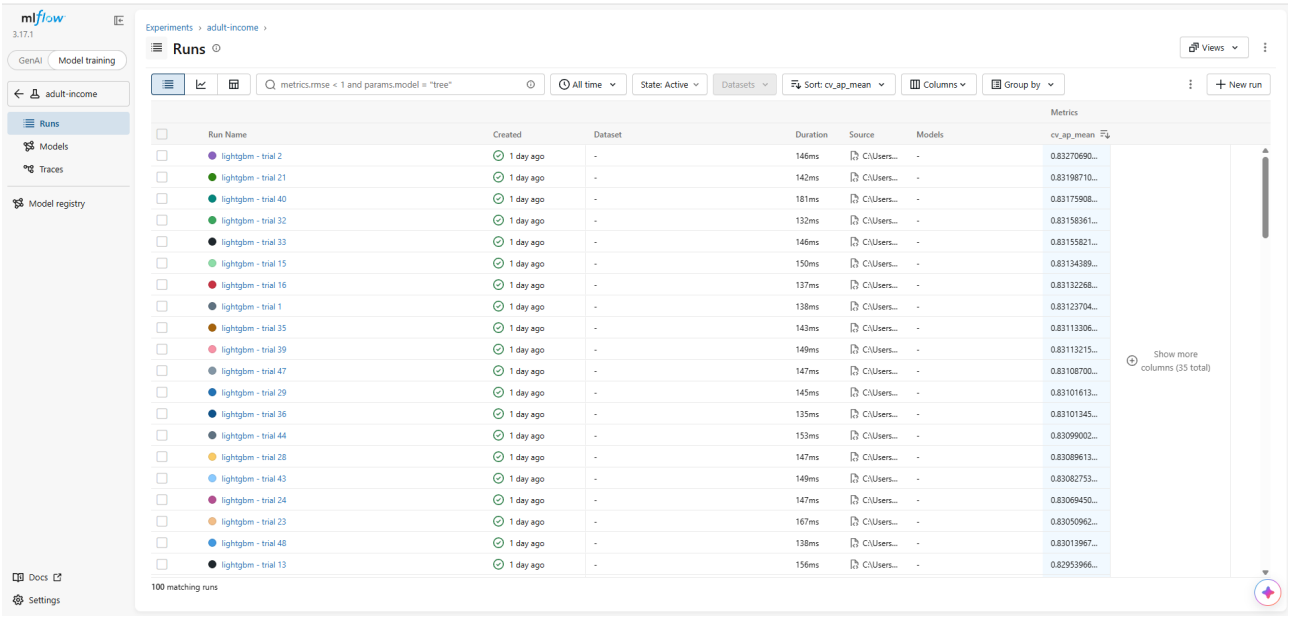


Figure 12: MLflow: the trials of the LightGBM tuning study, sorted by mean CV AP (`cv_ap_mean`). Trial 2 is the best, with 0.833.

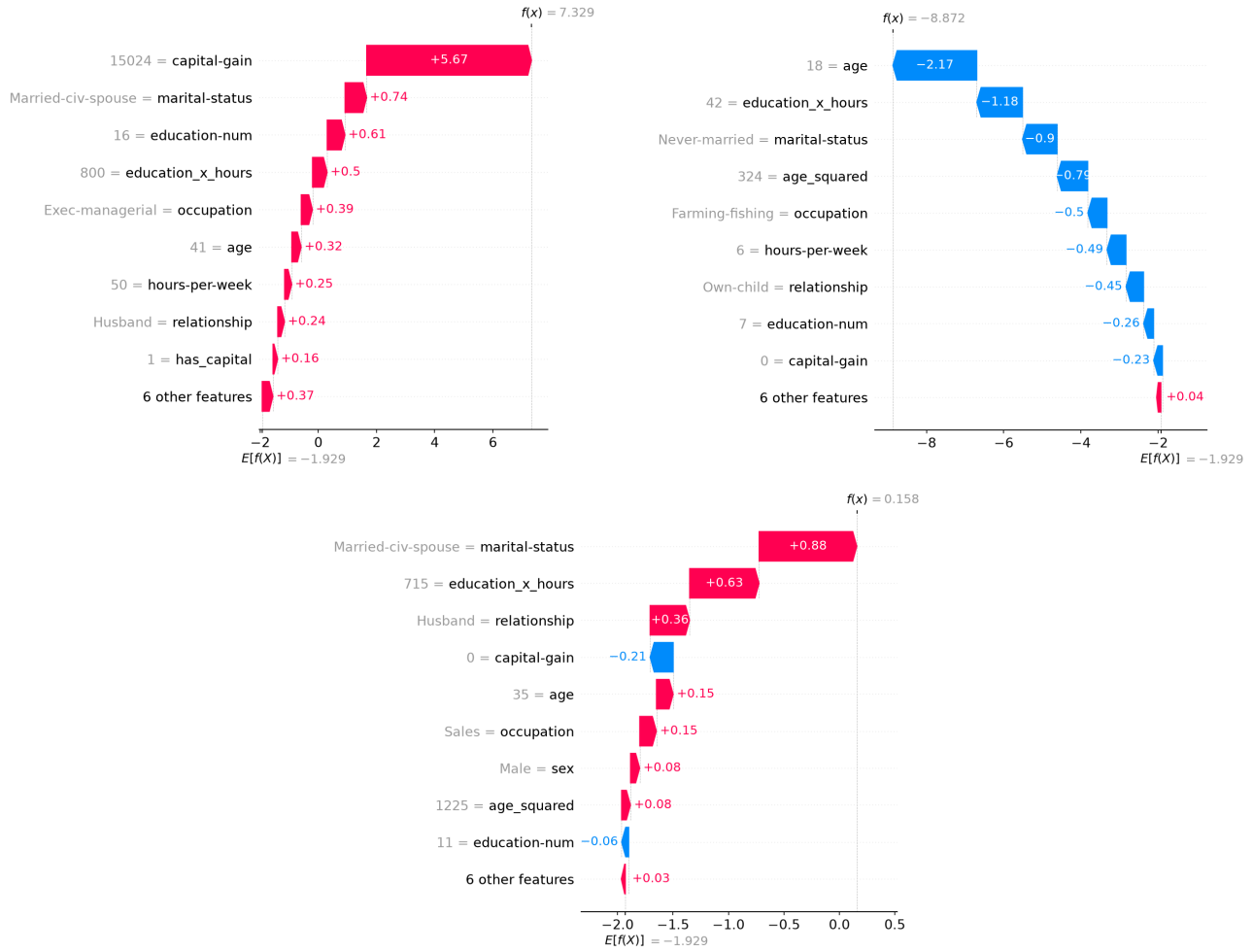


Figure 13: SHAP waterfall plots of the most confident >50K (top left), the most confident ≤50K (top right) and the person closest to the threshold (bottom): from the base value, each feature pushes the score up (red) or down (blue) until the final prediction.